Национальный исследовательский университет
ИТМО

Факультет Программной Инженерии и
Компьютерной техники

Информационные системы

Лабораторная работа 1

Выполнил:

Кузьмин Дмитрий Анатольевич

Группа:

P3309

Преподаватель:

Николаев Владимир Вячеславович

Санкт-Петербург 2026

1. UML-диаграммы классов и пакетов

Предметная область работы — учёт драконов. Серверная часть написана на Java 17 с использованием Jakarta EE 10 (JAX-RS, CDI, Bean Validation, JPA/Hibernate) и поставляется архивом `lab1.war`. Данные хранятся в PostgreSQL. Клиент — статические страницы, которые вызывают REST API по пути `/api` и получают сигнал об изменениях через WebSocket `/ws/dragons`.

1.1. Диаграмма пакетов

Классы разложены по пакетам в соответствии со слоями. Пакет `lab.rest` принимает HTTP-запросы. Пакет `lab.service` содержит сценарии и проверки. Пакет `lab.repository` обращается к базе. Пакет `lab.entity` описывает сохраняемые объекты. Пакет `lab.dto` задаёт тела запросов и ответов и не зависит от сущностей. Пакет `lab.mapper` собирает `DragonResponse` из сущности. Пакет `lab.ws` рассылает клиентам уведомление после изменений.

Стрелка на рисунке 1 означает зависимость: пакет на стороне начала стрелки импортирует классы пакета, на который она указывает.

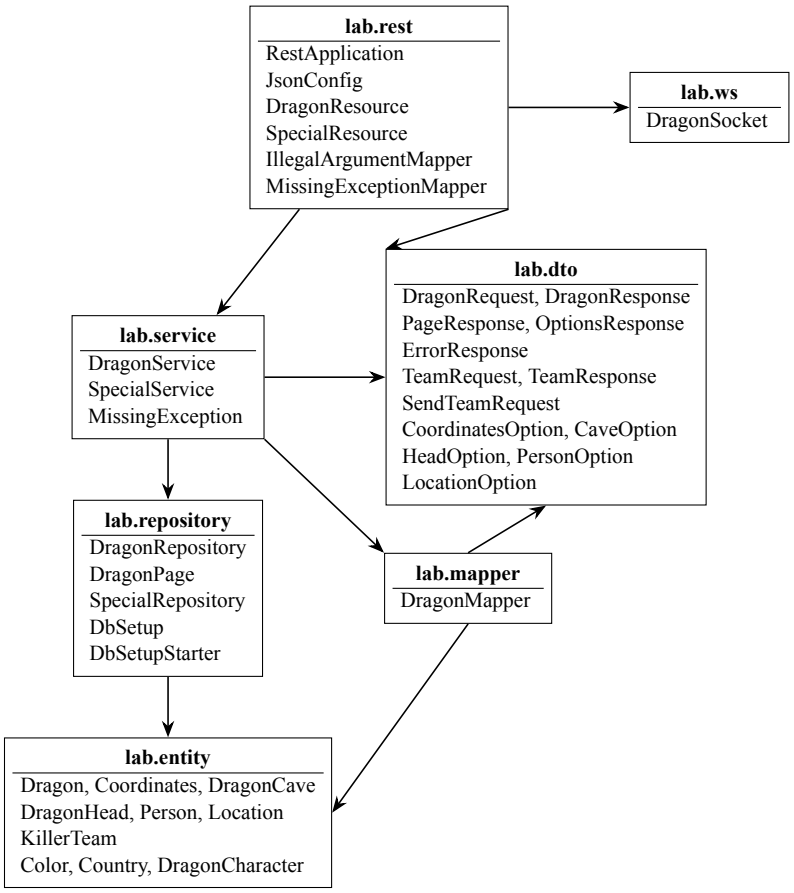


Рис. 1 – Диаграмма пакетов серверной части

1.2. Диаграмма классов предметной области

На рисунке 2 показаны сущности JPA и перечисления. Сплошная линия — ассоциация, подписи задают кратность. Пунктир — использование перечисления. Общие объекты (`Coordinates`, `DragonCave`, `DragonHead`, `Person`, `Location`) могут быть связаны с несколькими драконами или людьми, поэтому связь не является композицией.

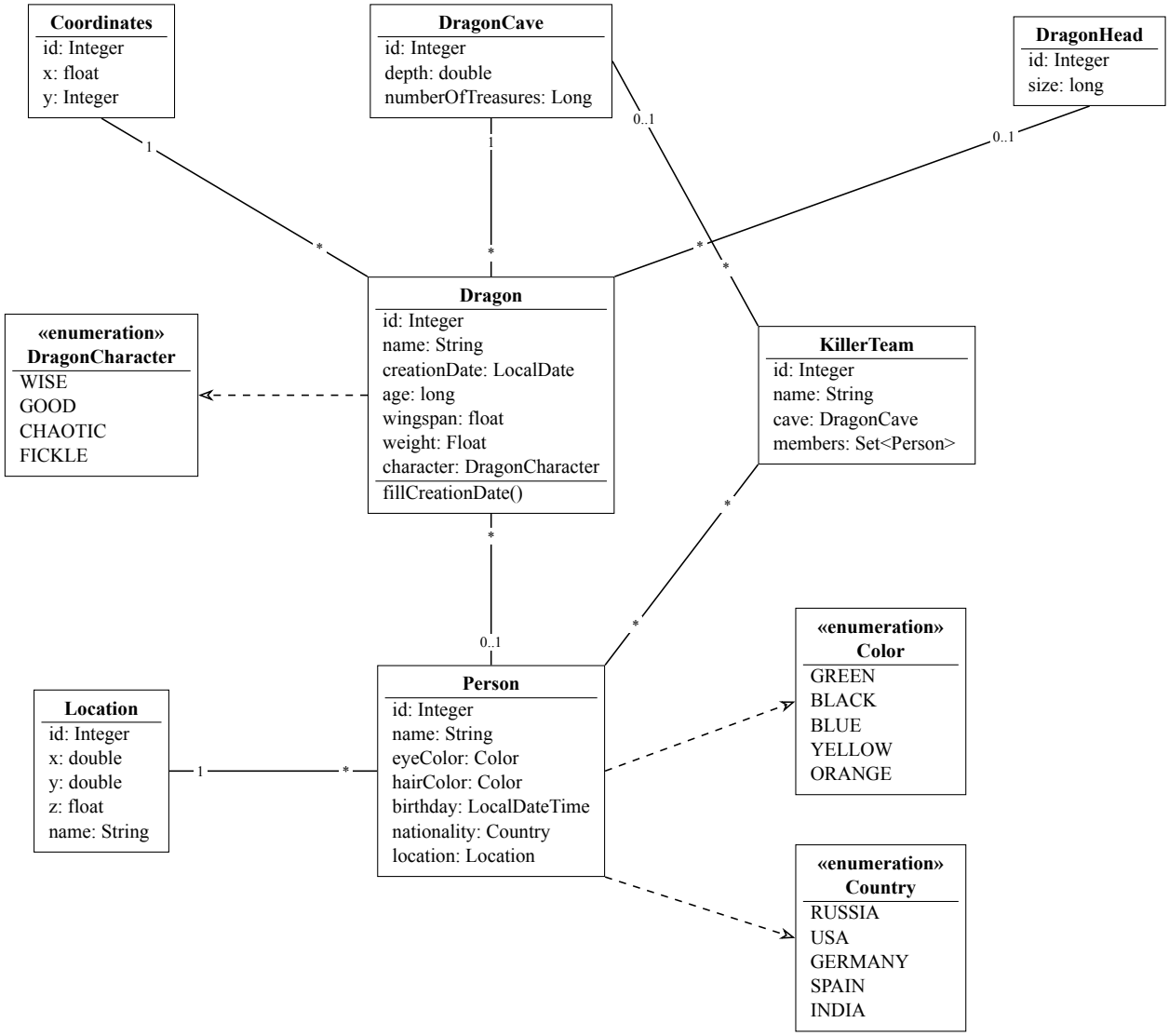


Рис. 2 – Диаграмма классов предметной области

`Dragon` обязательно ссылается на координаты и пещеру. Голова и убийца могут отсутствовать. Дата создания проставляется перед первой записью, если клиент её не передал. `Person` обязательно имеет местоположение; цвет глаз обязателен, цвет волос и гражданство — нет. `KillerTeam` объединяет убийц связью «многие ко многим» и может быть направлена в пещеру.

1.3. Диаграмма классов приложения

На рисунке 3 показаны классы, через которые проходят запросы. Ресурсы делегируют работу сервисам и после создания, изменения, удаления дракона, а также после операций с командой вызывают `DragonSocket.broadcast()`. Сервисы открывают транзакцию, проверяют данные и обращаются к репозиториям. `DragonMapper` переводит сущность в ответ API. Если объект не найден, выбрасывается `MissingException`; обработчики в `lab.rest` превращают её и ошибки проверки в JSON.

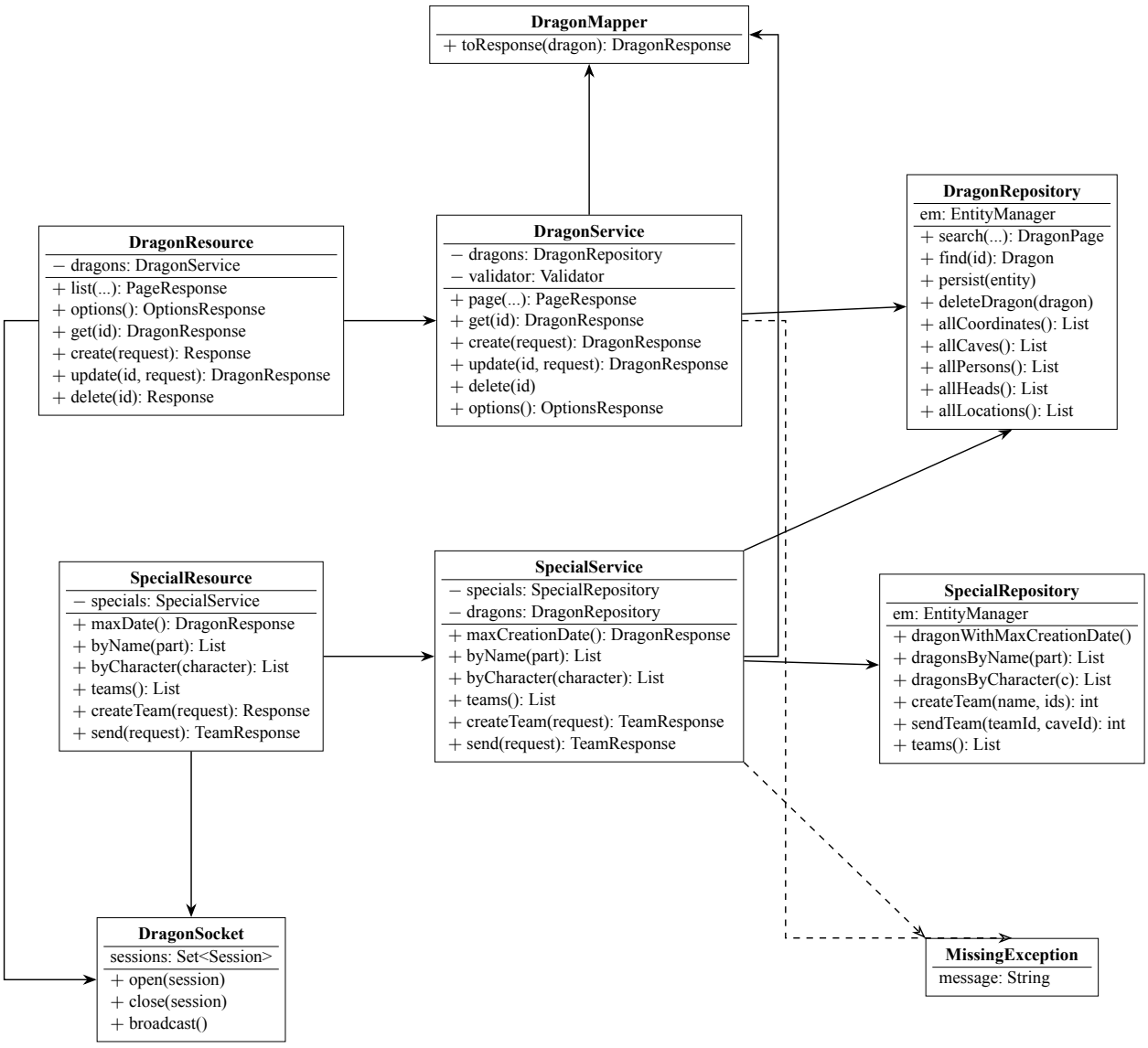


Рис. 3 – Диаграмма классов прикладного слоя

Обычный сценарий дракона: страница списка с номером страницы, размером, сортировкой и фильтром по полю; чтение одной записи; создание; изменение; удаление. Справочник `options` отдаёт уже сохранённые координаты, пещеры, головы, людей и места, чтобы их можно было выбрать повторно. Специальные операции: дракон с наибольшей датой создания, поиск по части имени, поиск по характеру, список команд, создание команды из выбранных убийц и отправка команды в пещеру.

2. Исходный код

Исходный код системы расположен в репозитории на ветке `for_gelios`:

https://github.com/Dmitry-Kuzmin456/is_lab_1/tree/for_gelios

3. Выводы

В работе построена информационная система учёта драконов. Модель данных покрывает дракона и связанные с ним координаты, пещеру, голову и убийцу, а также команду убийц, которую можно собрать и направить в пещеру. Ограничения предметной области заданы и в сущностях, и при разборе запроса, поэтому в базу не попадают пустое имя, неположительные возраст, вес, размах крыльев и число сокровищ, неизвестные значения перечислений.

Разделение на пакеты отделяет контракт API от хранения. Ресурсы не собирают SQL и не заполняют сущности, репозитории не знают формат JSON, а объекты `dto` не тянут за собой таблицы. За счёт этого постраничный список, фильтр и специальные выборки добавляются в своём слое, не переписывая остальные. Общие координаты, пещеры, головы и люди не копируются при каждом драконе: запись ссылается на уже существующий объект или создаёт новый, а при удалении неиспользуемый объект убирается.

Изменяющие операции сообщают открытым страницам о необходимости обновить данные. Так список и экран специальных операций остаются согласованными между несколькими открытыми клиентами без периодического опроса сервера.